Ray Data: The streaming batch model for heterogeneous data processing

Stephanie Wang
UW Allen School

RAY

Outline

1. Why heterogeneous data processing?
2. Challenges with existing batch and stream processing models
3. The streaming batch model and Ray Data

Why heterogeneous data processing?

Machine learning (surprise!) is:

- GPU-intensive *and* CPU-intensive
- Diverse and complex in its preprocessing requirements
- Vulnerable to CPU failures at large scale

Why heterogeneous data processing?

Ray Data:

- Allows independent scaling of CPUs and GPUs, to leverage **heterogeneous clusters**
- **Dynamically reconfigures** resources per operator, adapts to memory pressure
- Recovers from **CPU failures** with **minimal impact** on GPU execution, and vice versa

Why heterogeneous data processing?

Ray Data:

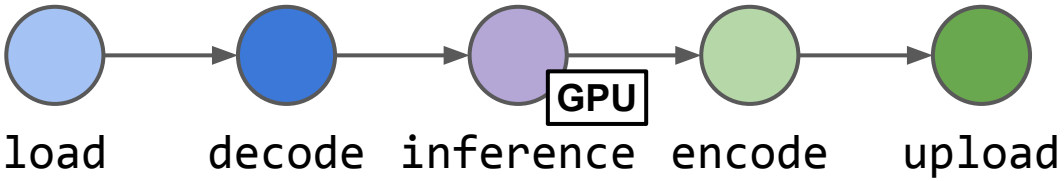
- A key part of today's Ray ecosystem (www.ray.io): Ray is a distributed compute engine for scaling ML apps
- Popular for **batch inference**, **data loading** for distributed training, especially for **multimodality**
- Used by: Pinterest, Instacart, NVIDIA, ByteDance, DataComp-LM, …

Heterogeneous data processing in ML pipelines

Batch inference

Load an image dataset from S3, apply a model, and upload the results

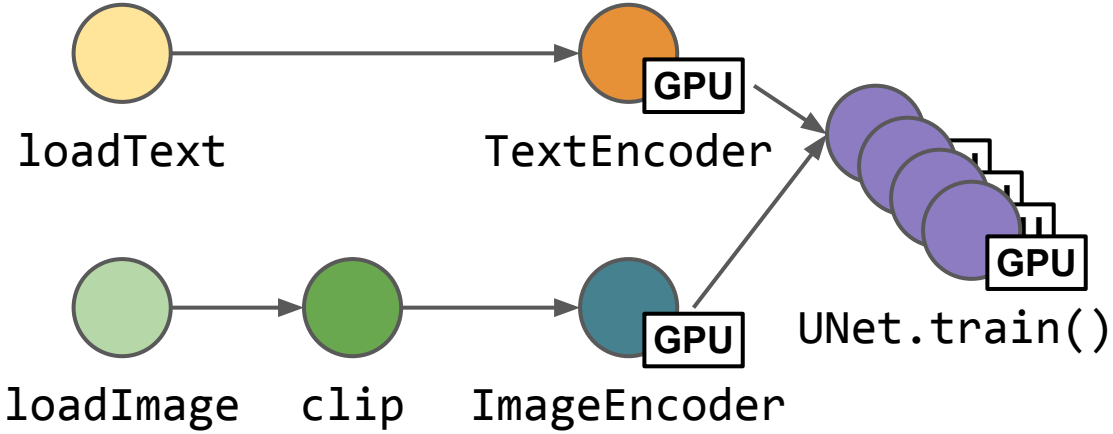
→ **3-8x higher throughput than Apache Spark and Flink**



Training Stable Diffusion

Load text and image dataset, encode each, and then partition among data-parallel trainers

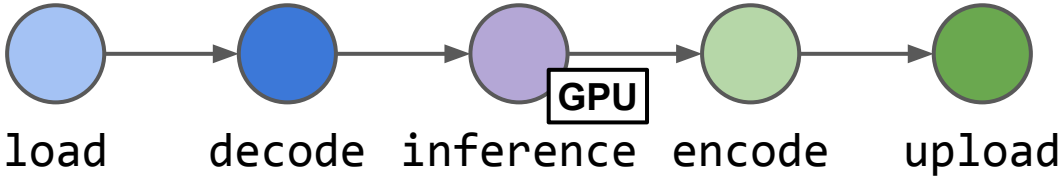
→ **44% higher throughput than ML-specific data loaders**



Requirements

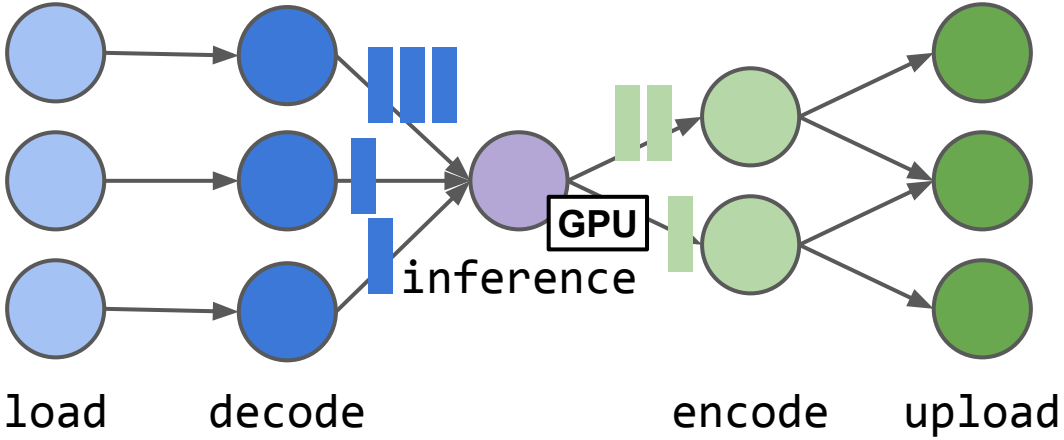
1. Each operator may require different physical parallelism.

   # CPUs >> # GPUs



Requirements

1. Each operator may require different physical parallelism.

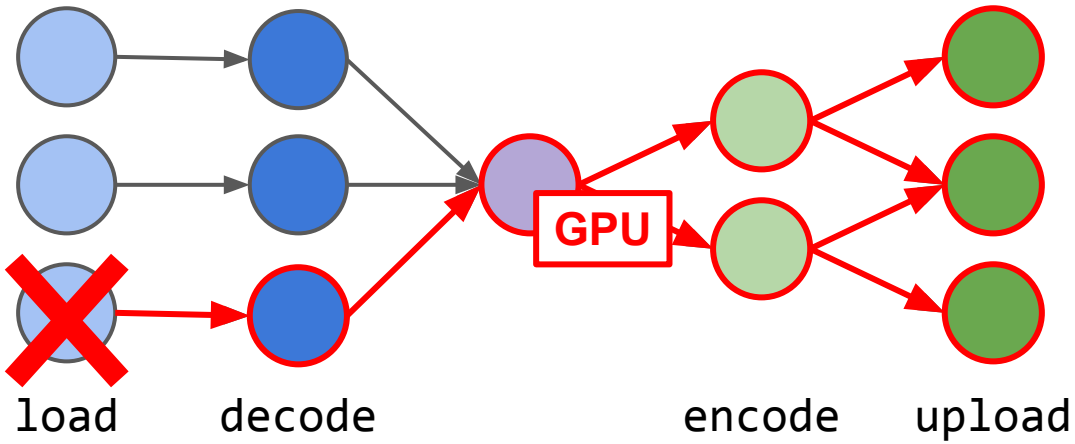


Decouple the physical parallelism of each operator

How do we manage memory for intermediate data?

Requirements

2. Different resources have different cost.



Efficient recovery and re-scaling of physical resources.

How do we minimize run-time overheads?

Requirements

1. Decouple the physical parallelism of each operator.

   → *How do we manage memory for intermediate data?*

2. Efficient recovery and re-scaling of physical resources.

   → *How do we minimize run-time overheads?*

Challenge: Dynamically reconfigure each operator's resources, while minimizing run-time and recovery overheads.

“Traditional” distributed data processing

Batch processing systems: MapReduce, Apache Hadoop, Apache Spark, …

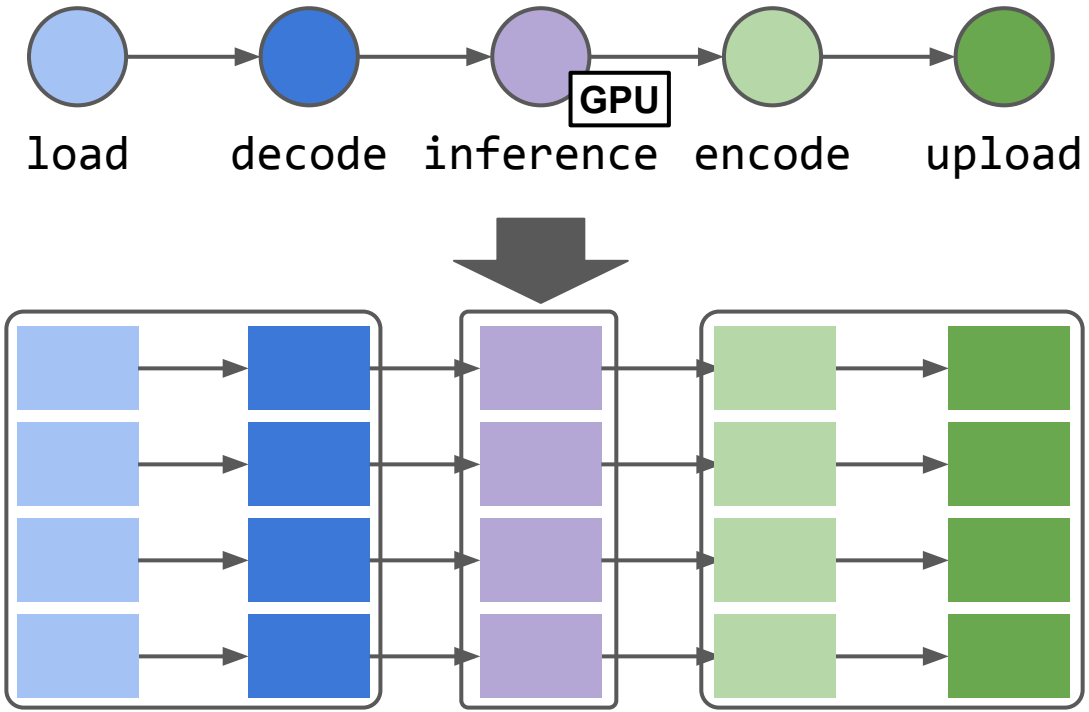
Stream processing systems: Apache Flink, Naiad, Google’s Millwheel, …

These systems have different limitations in dynamic reconfigurability and run-time vs. recovery overheads.

Key idea: Stream + batch = Streaming batch

Batch processing model

Goal: Allow a task to run and therefore recover on any executor.

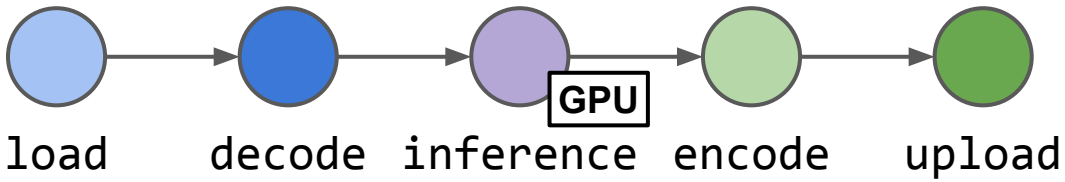


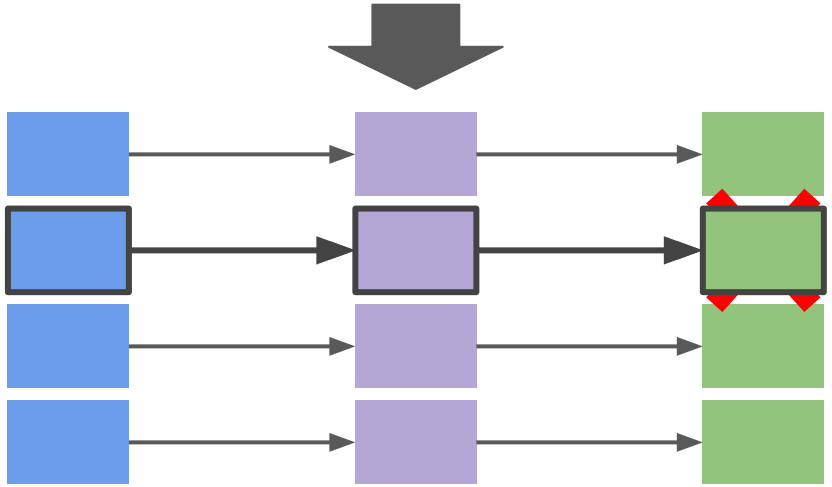
Partition each operator into a stage of data-parallel **stateless tasks**.

Fuse tasks with the same resource requirement to reduce memory usage.

Batch processing model

Goal: Allow a task to run and therefore recover on any executor.



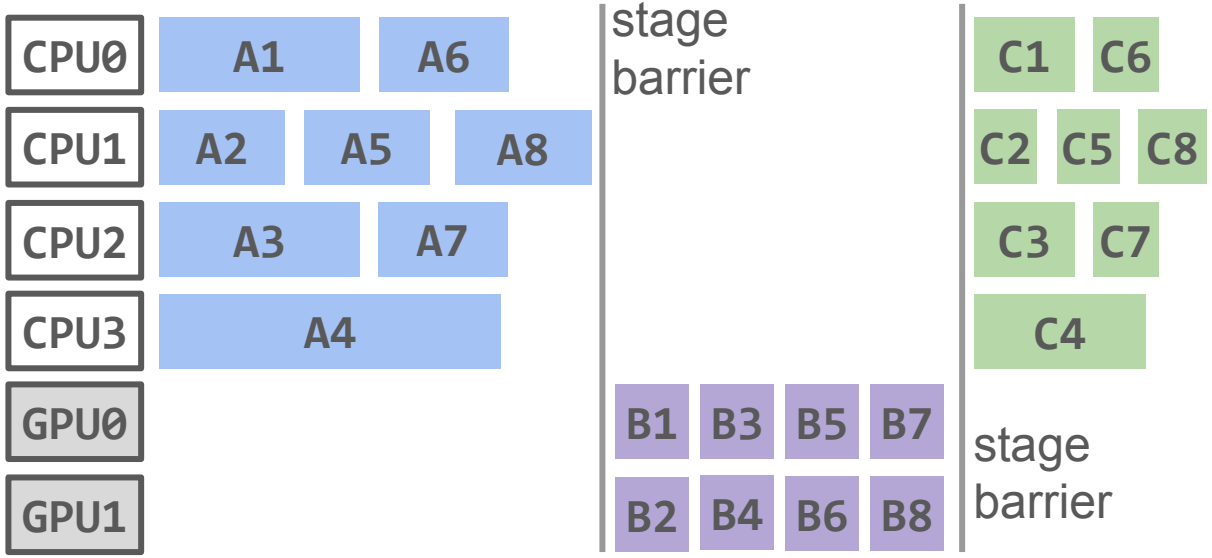


+ Low recovery overhead
+ Elasticity

Re-execute the data partition's **lineage**

Batch processing model

Goal: Allow a task to run and therefore recover on any executor.



Each stage must finish executing *before* next begins.

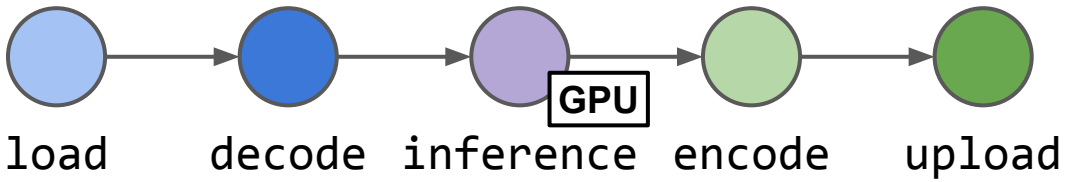
→ *No pipelining; high materialization overhead*

Number of partitions decided *before* run time.

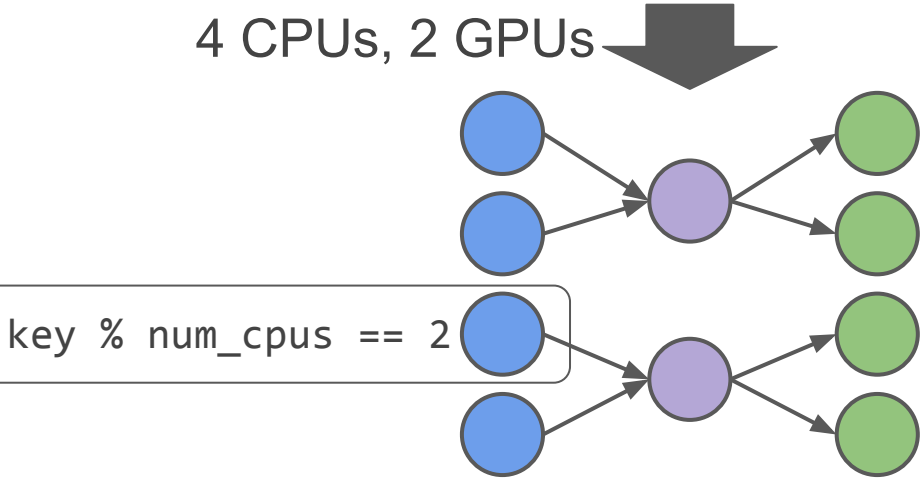
→ *Cannot adapt to memory pressure*

Stream processing model

Goal: Reduce latency through asynchronous execution



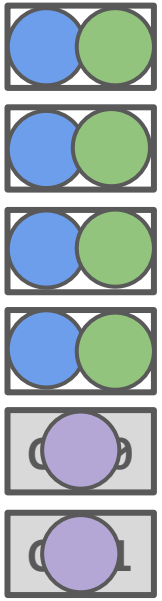
4 CPUs, 2 GPUs



Partition each operator into a group of data-parallel **stateful operators**.

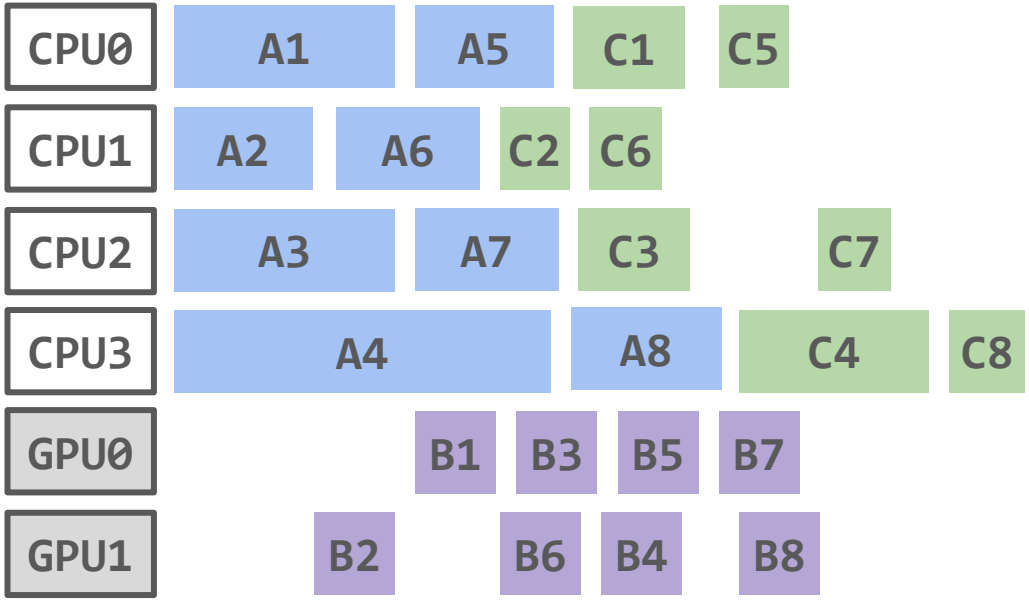
Stream processing model

Goal: Reduce latency through asynchronous execution



Stream processing model

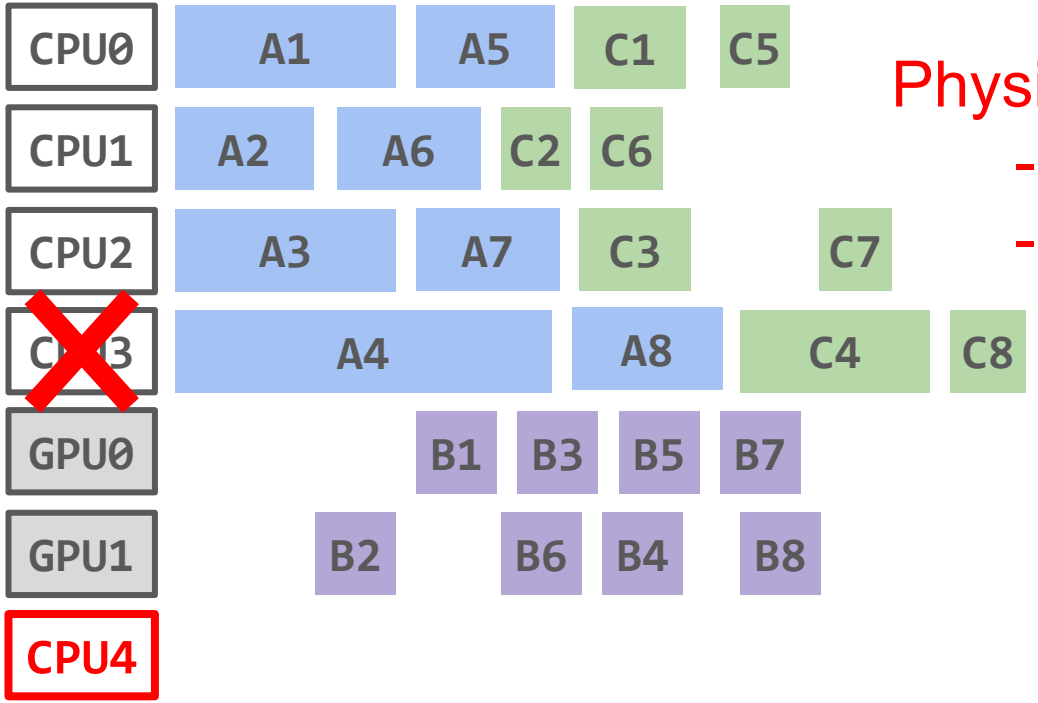
Goal: Reduce latency through asynchronous execution



+ Pipeline execution between resources
+ Executors adapt to memory pressure

Stream processing model

Goal: Reduce latency through asynchronous execution

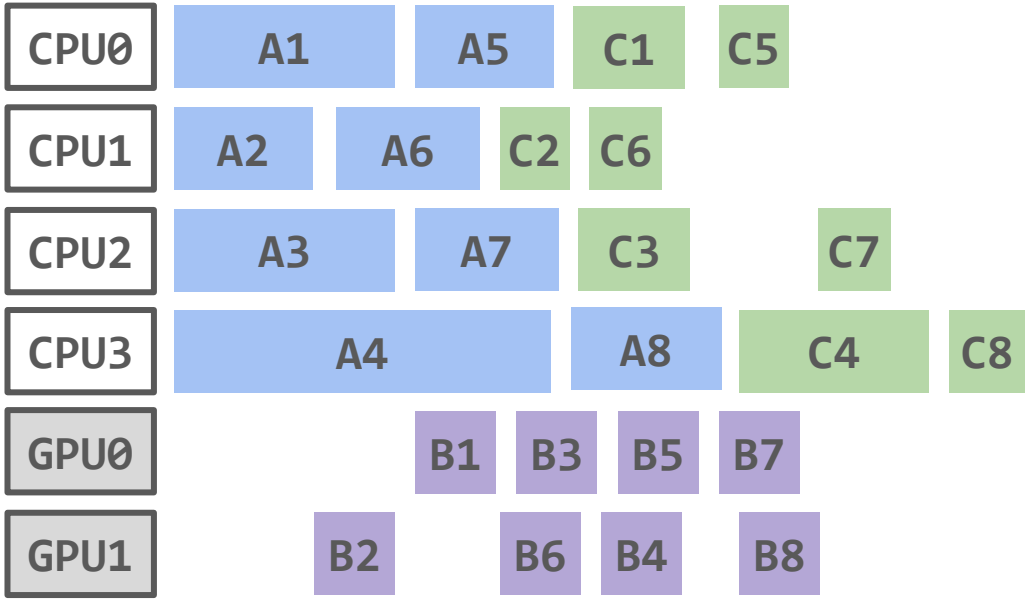


Physical reconfiguration:

- Global checkpointing vs.
- Logging

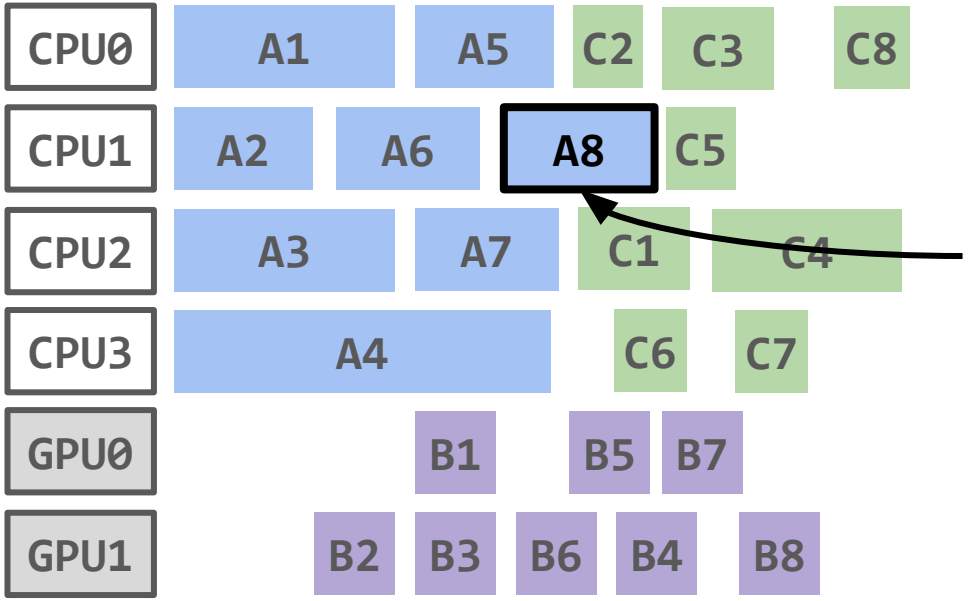
Stream processing model

Goal: Reduce latency through asynchronous execution
+ allow any task to run on any executor (with the matching resources).



Streaming batch model

Goal: Reduce latency through asynchronous execution
+ allow any task to run on any executor (with the matching resources).



Fast reconfiguration: Executor is not assigned to a range partition. Tasks can run on any CPU executor.

Streaming batch model

Key idea: Combine the batch and stream processing models.

- **Batch:** Centralized scheduler, stateless tasks, lineage recovery

  → Low recovery and reconfiguration overhead

- **Stream:** Asynchronous stages, local and dynamic repartitioning

  → Pipelined execution, reduce and adapt to memory pressure

Challenges:

1. User specifies only the *partition size*. The number of partitions is not known until run time, but the lineage must be recorded *before* execution.
2. Centralized scheduler must balance compute utilization vs. memory usage.

Streaming batch model

Key idea: Combine the batch and stream processing models.

- **Batch:** Centralized scheduler, stateless tasks, lineage recovery

  → Low recovery and reconfiguration overhead

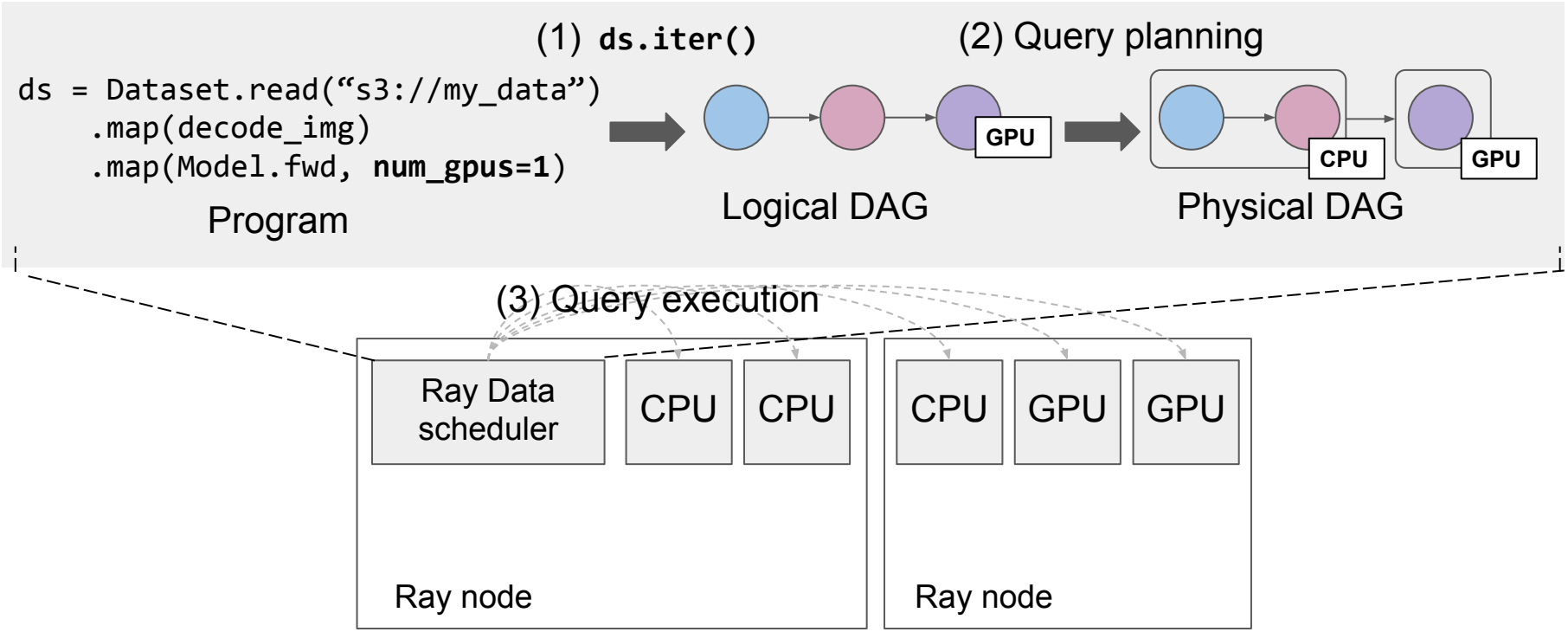
- **Stream:** Asynchronous stages, local and dynamic repartitioning

  → Pipelined execution, reduce and adapt to memory pressure

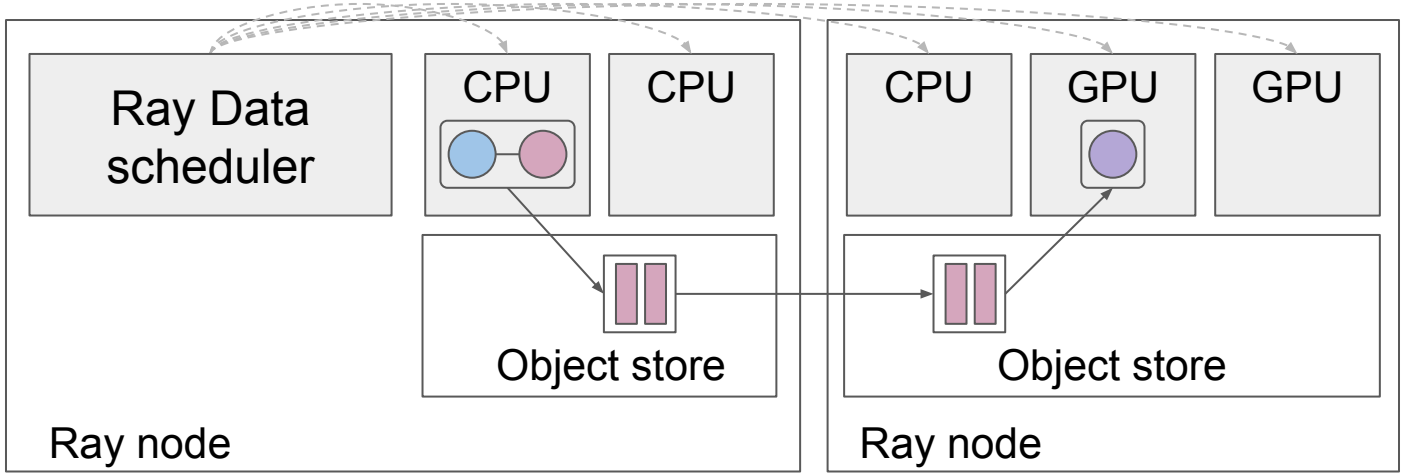
Challenges:

1. **User specifies only the *partition size*. The number of partitions is not known until run time, but the lineage must be recorded *before* execution.**
2. Centralized scheduler must balance compute utilization vs. memory usage.

Ray Data Overview



Ray Data Overview: Query execution

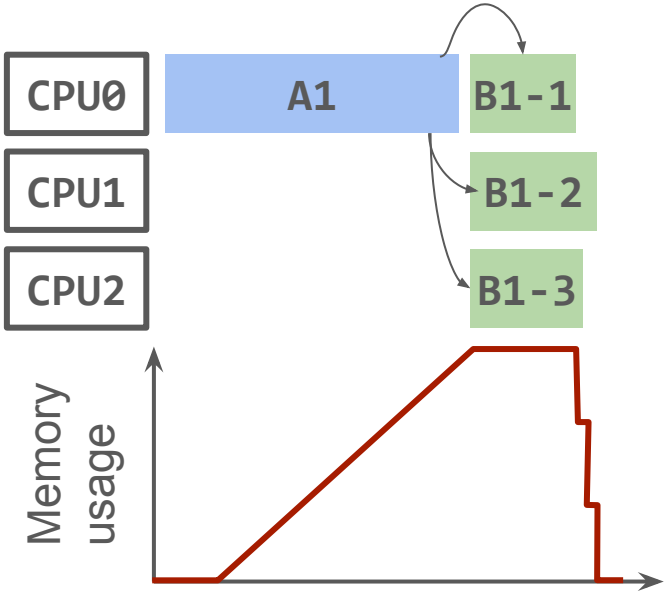


Ray Data scheduler uses Ray's dynamic dataflow interface:

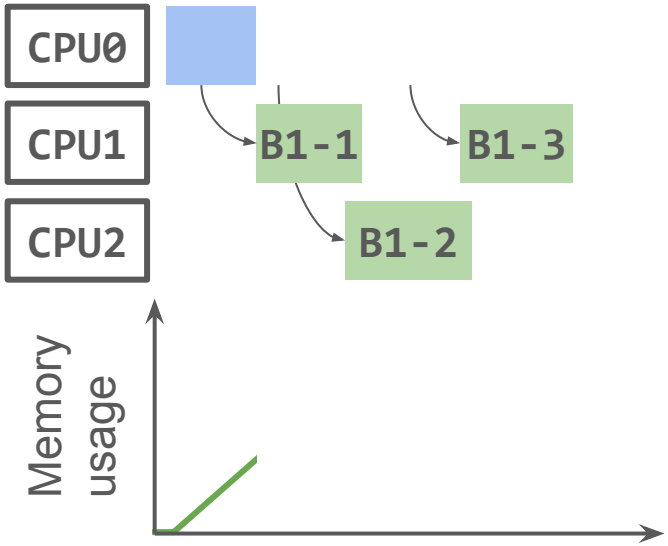
```
ref = load.remote(input_file)  # Returns dist future.
ref = map.remote(partition, ref)  # Specify data dep.
ray.wait(ref)  # Wait for task to finish.
```

Streaming repartition to limit memory usage

Repartition in Ray and batch processing systems



Streaming repartition in Ray Data



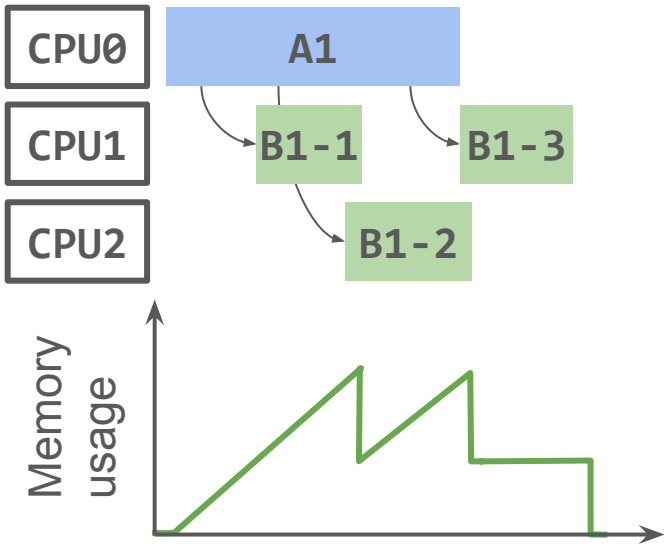
Streaming repartition to limit memory usage

Extend Ray with Python generators.

```
@ray.remote
def exec_B(data: Iter[Record]):
  part = []
  for row in data:
    part.append(fn(row))
    if sizeof(part) > N:
      yield part
  if part:
    yield part
```

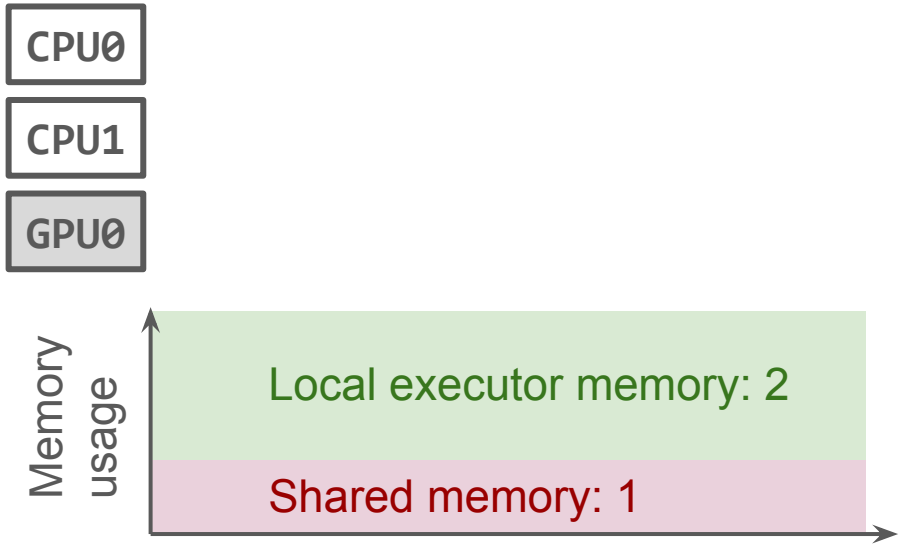
How to support lineage recovery?
Record number of partitions,
deterministic partitioning.

Streaming repartition in Ray Data

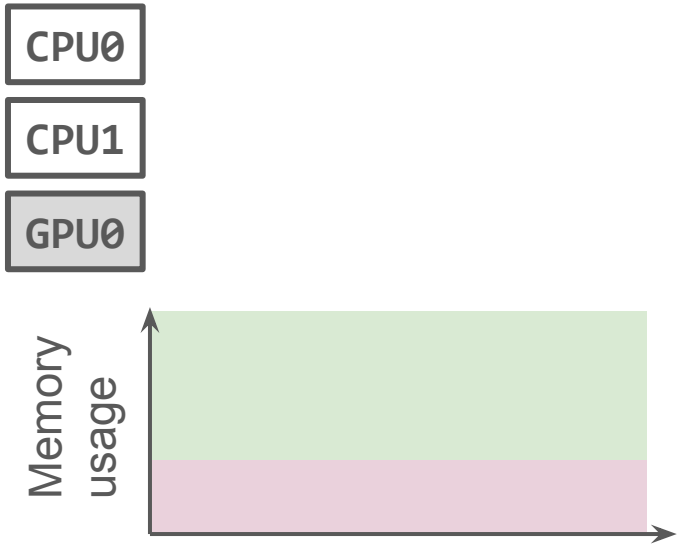


Adaptive scheduling with memory limits

Conservative scheduler

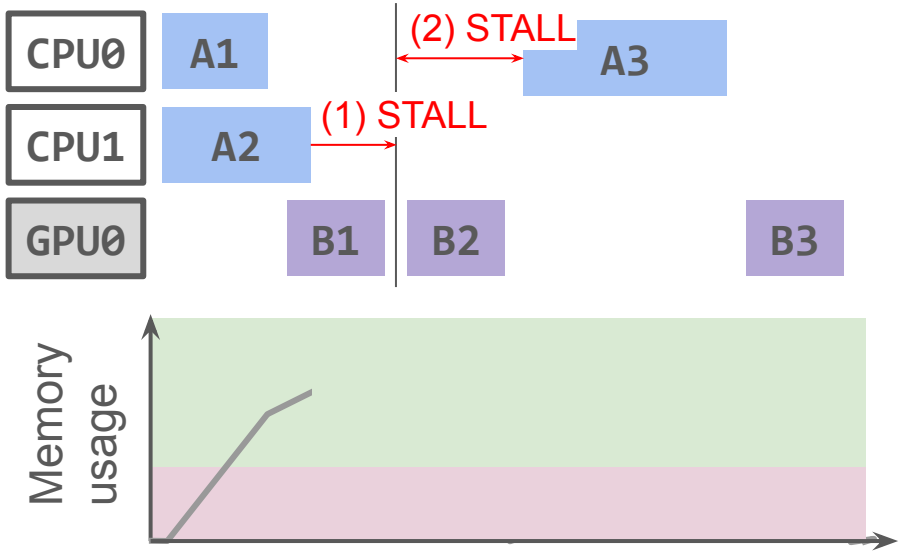


Optimistic scheduler

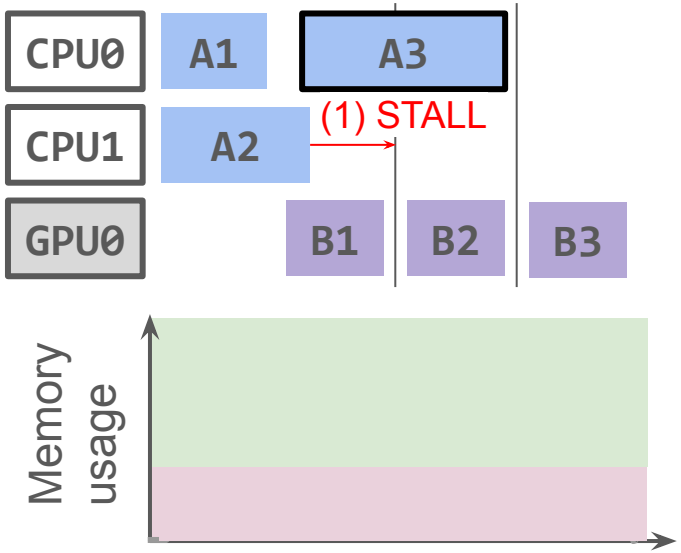


Adaptive scheduling with memory limits

Conservative scheduler



Optimistic scheduler



Scheduling policy

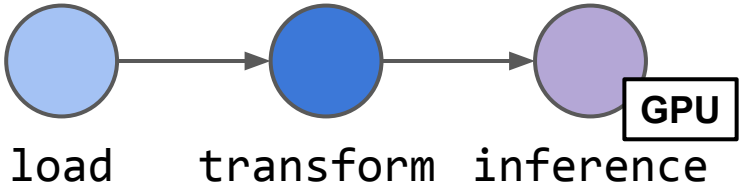
Conservative policy

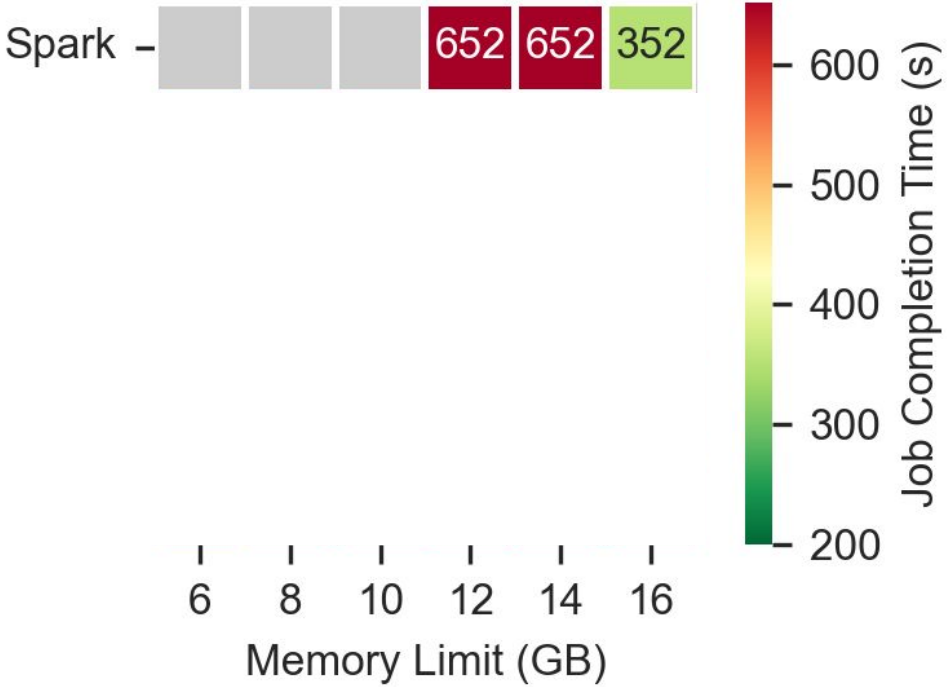
- Prioritize operators with shorter output queues
- Stall tasks when memory limit reached
- Equivalent to backpressure mechanism used in most streaming systems

Optimistic policy

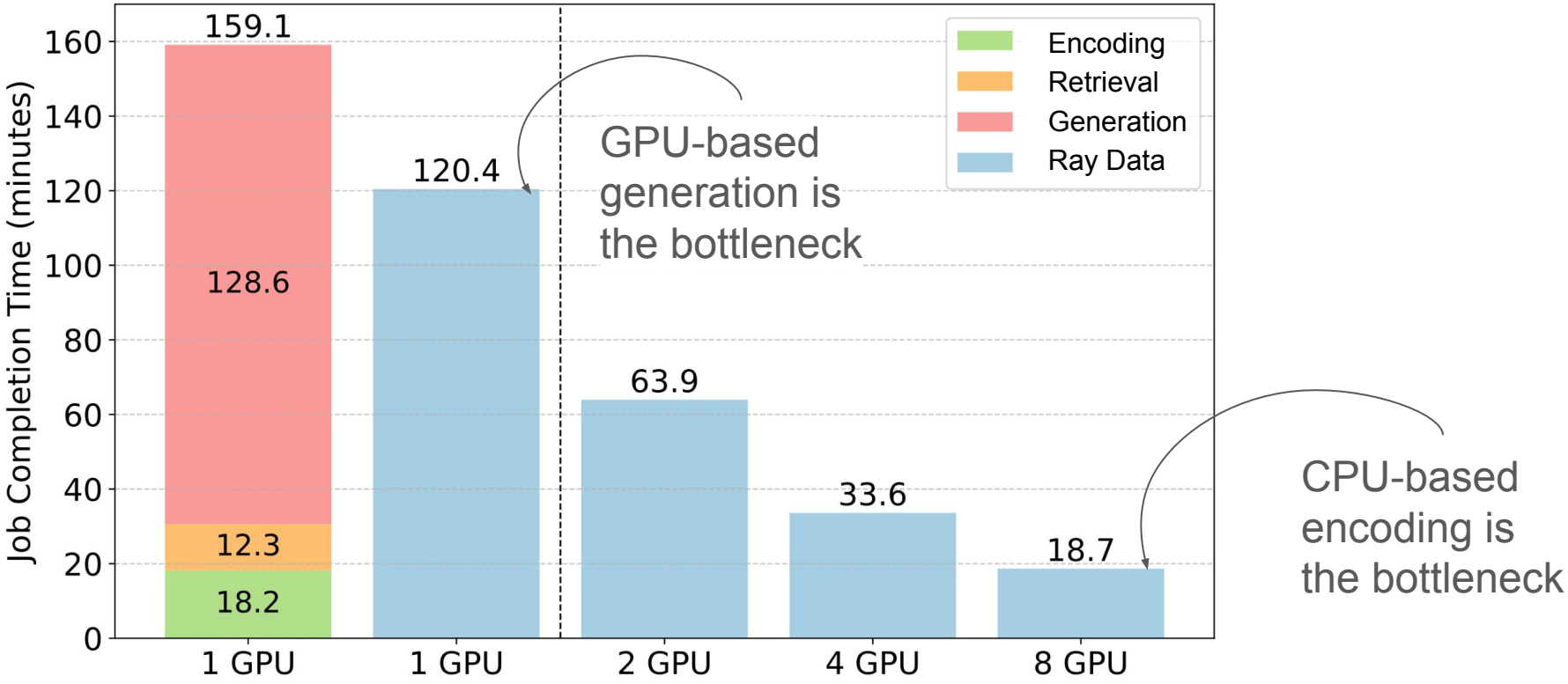
- Estimate each operator's throughput → total pipeline throughput → rate that memory is released
- Track global memory budget
  - Subtract when submitting *source* task
  - Add based on estimated memory release rate
- Not exact, but creates a positive feedback loop

Evaluation: Memory limits





Evaluation: Retrieval-augmented generation



Evaluation: Batch inference

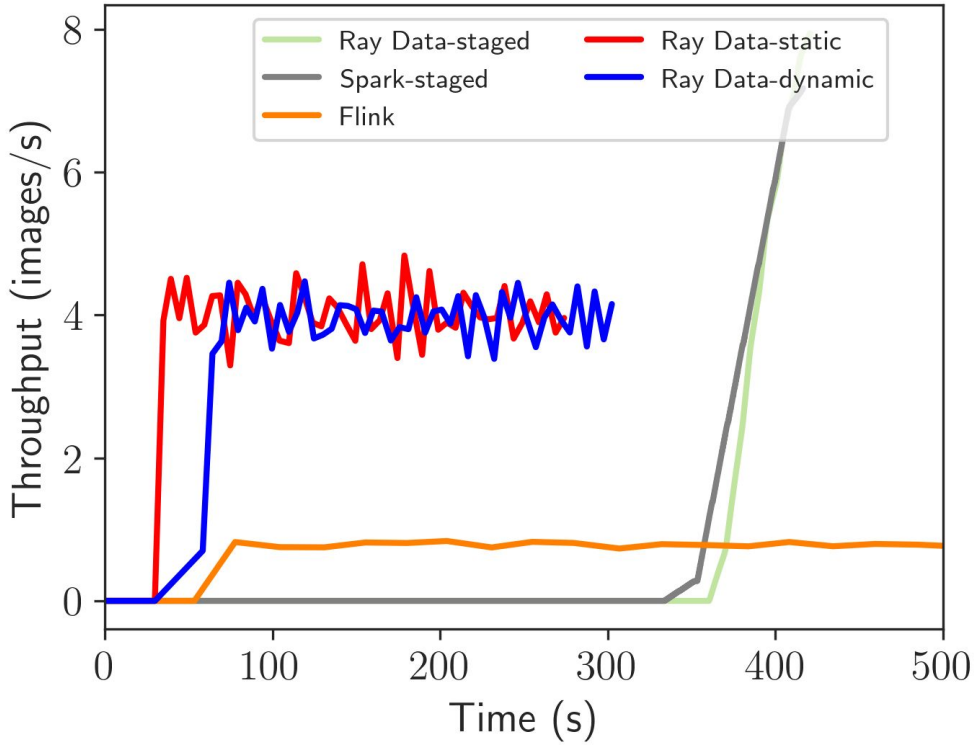
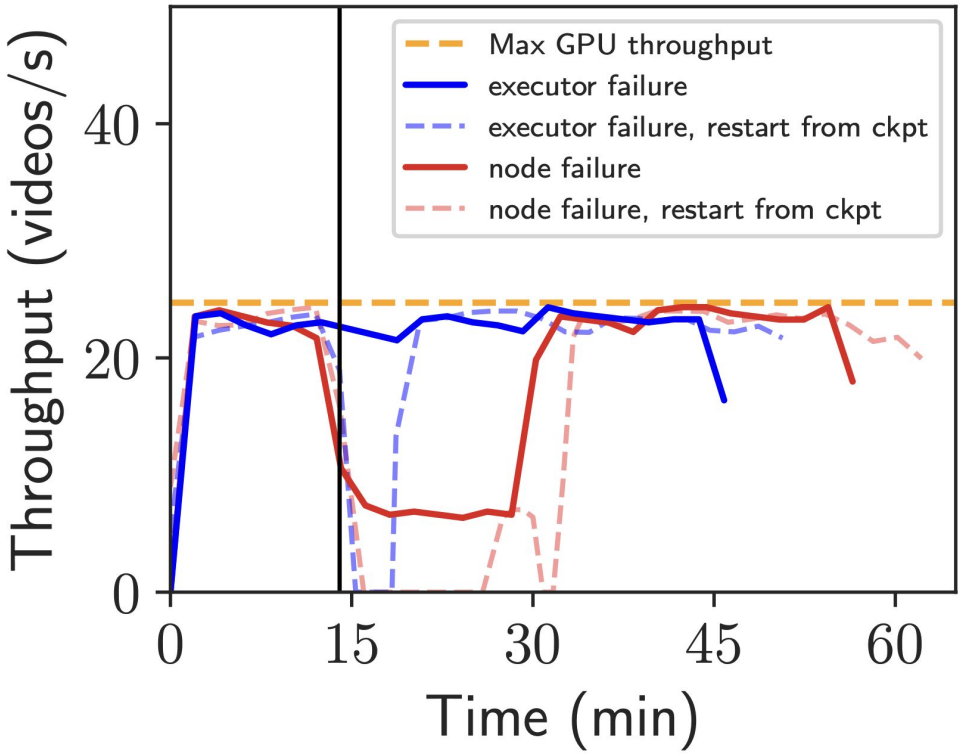


Image-to-image generation pipeline

1. Download images
2. Preprocess images: normalize, etc.
3. Generate (GPU): Stable Diffusion
4. Encode+upload

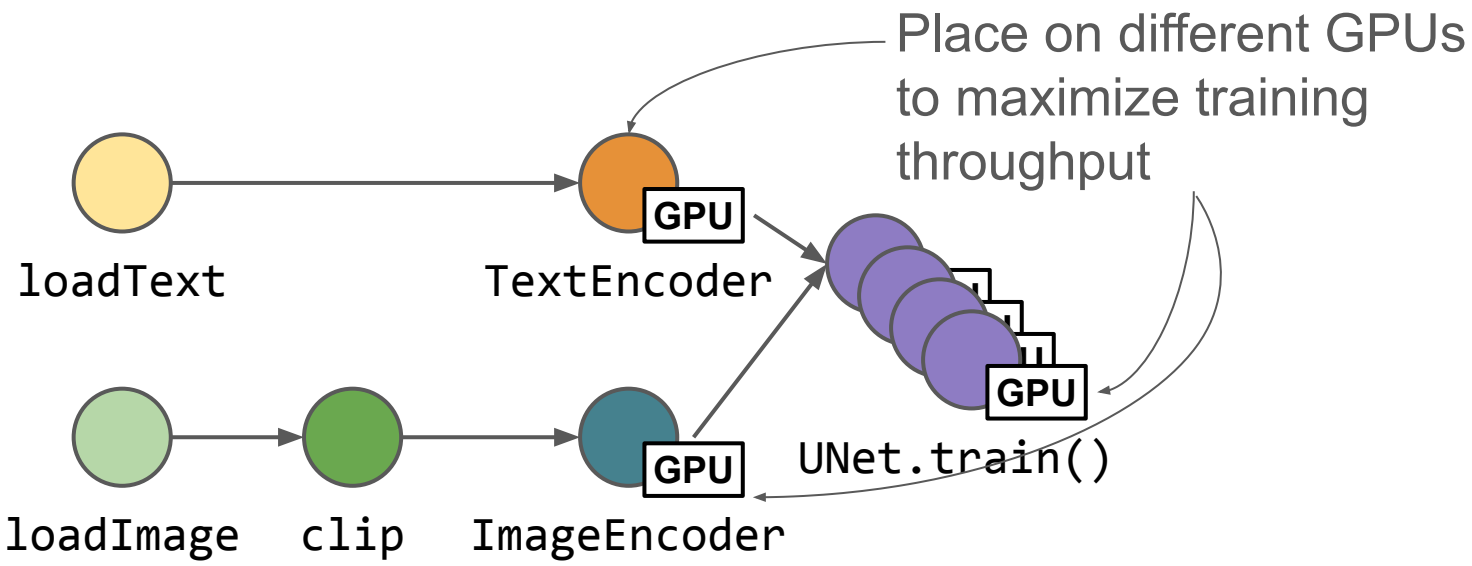
Evaluation: Elasticity



Video classification:

- Model: VideoMAE;
  Dataset: Kinetics-700-2020
  Cluster: 1 GPU node + 1 8-CPU node
- Executor failure: worker process failure
- Node failure: disconnect CPU-only node for 15min

Evaluation: Training Stable Diffusion



Evaluation: Training Stable Diffusion

	Resources	Images/s	Run time (hours)	Total cost
PyTorch DataLoader **(stream)**	4x p4de.24xlarge (8xA100)	2,811	111.3	$18,192

The Streaming Batch Model

Combine the batch and stream processing models to achieve efficient and fault-tolerant heterogeneous execution.

Future work

- Dynamic query (re)planning
- Autoscaling for heterogeneous clusters
- Distributed memory management and finer-grained orchestration for accelerators (at HotOS!)